

ProjectAlma - Lead Management System

Design Document

Author: Engineering Date: 2026-08-31 Status: Implemented

1. Overview

ProjectAlma is a lead-management application for an immigration law practice. It has three surfaces:

- A **public intake form** where a prospective client submits their name, email,

and resume. No account is required.

- An **internal dashboard** where an attorney reviews every lead, sees all

submitted information, and records when they have reached out.

- A **read-only applicant portal** where a prospect who creates an account can

track the status of the case(s) they submitted.

On every submission the system emails both the prospect (a confirmation) and an attorney (a notification). Each lead has a lifecycle: it begins in `PENDING` and an attorney moves it to `REACHED_OUT` once they have made contact; that move is reversible so a mistake can be corrected, and every change is recorded.

The system is a FastAPI backend and a Next.js frontend, with PostgreSQL for persistence, pluggable object storage for resumes, and a transactional email provider.

2. Goals and non-goals

Goals

- A frictionless, fully public intake path - the single most important flow.
- A trustworthy internal view: every field the prospect submitted, an accurate status, and an audit trail of who did what and when.
- A clean separation between the browser, the web tier, and the business tier so the session token is never exposed to client-side JavaScript.
- Production-shaped operability: structured logs, request correlation, health and readiness probes, a consistent error contract, rate limiting, and hardened file uploads.

- Every external dependency (database, file storage, email, rate limiting) behind

an interface so it can be swapped without touching business logic.

Non-goals (deliberately out of scope)

- Email verification on account registration, token revocation, and SSO - see

Section 11.

- Guaranteed email delivery (durable queue with retries and a dead-letter queue).
- Virus scanning of uploaded files.
- Full-text or ranked lead search, and role-based access control beyond the two

personas below.

- Real-time collaborative updates on the dashboard.

3. Requirements

Functional

Actor	Requirement
Prospect (public, no account)	Submit first name, last name, email, and a resume/CV file.
System	On submission, email the prospect a confirmation and an attorney a notification.
Attorney (authenticated)	See all leads with every submitted field and a status summary; open a lead; download its resume; mark it <code>REACHED_OUT</code> (and reopen it if needed).
Applicant (authenticated, optional)	Register an account and see the status, milestone, and history of the case(s) they submitted.

Lead lifecycle: `PENDING` → (an attorney acts manually) → `REACHED_OUT`. The reverse move is permitted; every transition is logged with actor and timestamp.

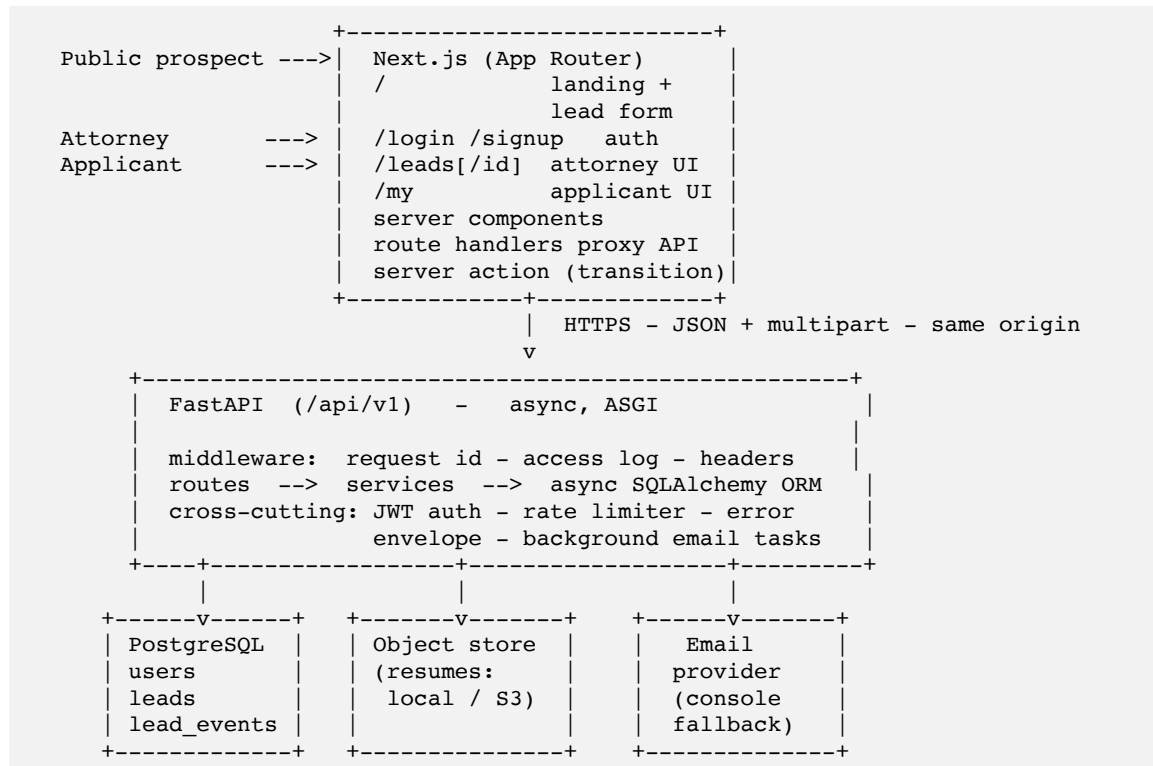
Non-functional

- The submission request must not block on, or fail because of, email delivery.
- Uploads must be validated by declared type, actual content, and size, with a bounded read so a large upload cannot exhaust memory.
- The public endpoints must be rate limited.
- Every non-2xx response must have a single, predictable JSON shape and carry a

correlation id that also appears in the logs.

- The application must run locally with no infrastructure (no database server, no message broker, no cloud credentials).

4. Architecture



Why the boundaries sit here

Next.js owns the browser session. The authentication token lives in an `httpOnly, SameSite=Lax` cookie set by a Next.js route handler. Browser JavaScript never sees the token or the API host. Server Components read the cookie and call the API server-side; the handful of browser-initiated calls (submit, login, resume download) go through thin Next.js route handlers that attach the token and forward the request.

All browser traffic is same-origin. Cross-origin request sharing is off the critical path, and the API host can be network-private in production.

FastAPI owns business rules and persistence and enforces them for every caller - authentication, the lead state machine, upload validation, and rate limits are server-side, not UI conventions.

The request path is fully asynchronous - ASGI, async SQLAlchemy, and an async Postgres driver - so a slow database call or a slow upload parks a coroutine rather than tying up a worker thread. The two unavoidably synchronous pieces (the S3 client and password hashing) run in a thread pool.

5. Component design

5.1 Frontend (Next.js, App Router, TypeScript)

- **List views are Server Components.** Filter, search, and pagination state live

in the URL query string; navigating re-renders the server component with fresh data. There is no client-side data store and no fetch-on-mount, so there is no client/server cache to reconcile and views are shareable and back-button-correct.

- **The one mutation** - moving a lead between states - is a Server Action behind

a confirmation dialog, followed by revalidation and a refresh.

- **Column sort**, the only genuinely ephemeral view state, is client-side within the current page.

- **Forms** use react-hook-form with zod schemas that mirror the server's own validation rules, so the user gets fast feedback while the server stays the authority.

- **Styling** is CSS Modules with design tokens - no CSS framework - because the UI is small and bespoke.

- **Route protection** is layered: middleware does a cookie-presence redirect for

/leads/* and /my/*; the route layouts then enforce the role (an applicant who reaches an attorney route is bounced to their portal, and vice versa). The API is the real gate regardless.

5.2 Backend (FastAPI)

Layered as routes -> services -> models:

```
core/      configuration (with a production boot guard), logging, security,
           error types, rate limiter, request context
db/        declarative base, async engine and session
models/    Lead, LeadEvent, User
schemas/   request and response models, generic Page[T], error response
services/  leads, users, email, storage (no web-framework imports; async; unit-
           tested)
api/       dependencies, middleware, one thin route module per resource
main.py    application factory, lifespan (seed, dispose engine), exception handlers
```

The service layer contains all business logic and knows nothing about HTTP, which keeps it directly unit-testable and keeps the edges swappable.

5.3 Data model

Managed by Alembic migrations. All identifiers are UUIDs.

``leads``

Column	Type	Notes
first_name, last_name	varchar	required, trimmed
email	varchar, indexed	required, lower-cased
resume_key	varchar	opaque key into the storage backend
resume_filename	varchar	sanitized original name
resume_content_type	varchar	from the allow-list
resume_size_bytes	int	
state	varchar(32), indexed	PENDING or REACHED_OUT
reached_out_at	timestamptz, nullable	cache of the current reached-out fact
reached_out_by_id	uuid, nullable, → users	who last marked it
created_at, updated_at	timestamptz	server-managed

``users`` - email (unique), full_name, hashed_password (bcrypt), role (attorney or applicant), is_active. An applicant is linked to their leads by matching email, not by a foreign key, so a prospect can submit first and register later.

``lead_events`` - an append-only audit trail.

Column	Type	Notes
lead_id	uuid, indexed, → leads (cascade)	
type	varchar(32)	CREATED, STATE_CHANGED, or EMAIL_SENT
message	varchar(512)	e.g. PENDING → REACHED_OUT
actor_id	uuid, nullable, → users	null for public or system actions
created_at	timestamptz (microsecond precision, set in code)	ordering key

Every lead creation and every state change writes an event **in the same database transaction** as the change itself, so the trail cannot drift from reality. The `reached_out_*` columns on `leads` are a convenience cache for the common "current status" read; the event log is the system of record for history.

5.4 Resume storage

An async `StorageBackend` interface with two implementations: a local-filesystem backend for development and a Docker volume, and an S3-compatible backend for production (object-store calls run in a thread pool; downloads can use presigned URLs). The implementation is selected by

configuration. Resumes are only ever served through the authenticated download endpoint, never via a public URL.

5.5 Email

An `EmailProvider` interface with a real transactional provider (used when an API key is configured) and a console provider that logs the message (the default, so development and CI run with no network). Each message is built by a pure function returning subject, HTML, and text. Both messages for a submission are dispatched from a background task that runs after the response is sent; the provider swallows and logs its own errors, so email can never fail or slow a submission.

6. Key flows

6.1 Lead submission

1. Validate the text fields.
2. Validate the resume: the declared content type is in the allow-list, a magic-byte check confirms the bytes match that type (this rejects a spoofed content-type header), the size is within the limit (the read is capped at the limit plus one byte so a huge upload cannot exhaust memory), and the filename is sanitized.
3. In one transaction: store the file via the storage backend, insert the lead in `PENDING`, and insert a `CREATED` audit event.
4. Register a background task for the two emails.
5. Return 201 with the new lead id and state.

6.2 State transition

A single service function is the only writer of `lead.state`. It validates the requested move against a transition table, rejects an illegal move with a 409, treats a no-op move as a successful 200, updates or clears the `reached_out_*` cache columns, and appends a `STATE_CHANGED` audit event naming the actor. The allowed moves are `PENDING -> REACHED_OUT` and `REACHED_OUT -> PENDING`; reopening appends a distinct `REACHED_OUT -> PENDING (reopened)` event rather than erasing anything.

6.3 Applicant portal

`GET /my/leads` returns the leads whose email matches the signed-in user's email, each with its embedded event history. The per-lead resume download returns 403 if the lead's email is not the caller's.

7. API reference (/api/v1)

Method	Path	Auth	Purpose
POST	/leads	public, rate-limited	Create a lead (multipart: fields plus resume).
GET	/leads	attorney	List; ?state= ? search= ?limit= ? offset=; returns items plus paging metadata.
GET	/leads/stats	attorney	Dashboard tiles: total, pending, reached-out.
GET	/leads/activity	attorney	Recent status changes across all leads, with lead and actor names.
GET	/leads/{id}	attorney	One lead with its embedded events.
GET	/leads/{id}/events	attorney	The lead's audit trail.
GET	/leads/{id}/resume	attorney	Download the stored resume.
PATCH	/leads/{id}	attorney	State transition, either direction.
GET	/my/leads	applicant	The signed-in prospect's own case(s), with events.
GET	/my/leads/{id}/resume	applicant	Download the caller's own resume (403 otherwise).
POST	/auth/login	public, rate-limited	Email and password, returns a token and the role.
POST	/auth/register	public, rate-limited	Register: applicant (open) or attorney (invite code required).
GET	/auth/me	any signed-in user	The current user, including role.
GET	/health	public	Liveness (no dependency checks).
GET	/ready	public	Readiness - pings the database, 503 if it is down.

Every lead payload also carries a computed **milestone** label derived from the state ("Awaiting attorney review" or "Attorney has reached out"), shown on both dashboards.

Error contract

Every non-2xx response has one shape:

```
{"error": {"code": "conflict", "message": "...", "details": [], "request_id": "..."} }
```

Domain errors are raised as typed exceptions in the service layer and mapped to a status and this envelope by a single set of handlers. A 429 carries a `Retry-After` header; every response carries an `X-Request-ID` (echoed from the request or generated), and that id appears in every log line for the request.

8. Security and authentication

- Password login issues a signed token (HS256) whose claims include subject,

issued-at, not-before, expiry, issuer, and a token type. Decoding requires all of them and verifies the issuer and type.

- The token is placed in an `httpOnly, SameSite=Lax` cookie by the Next.js

login handler. Server Components read it and call the API with a bearer header.

- `get_current_user` loads an **active** user or returns 401.

`get_current_attorney` additionally requires the attorney role (403 otherwise) and guards every internal endpoint; the applicant portal accepts any signed-in user.

- Login is rate limited and returns a generic message; a dummy password-verify

runs for unknown emails to blunt timing-based account enumeration.

- **Registration** takes a role. `applicant` is open (someone tracking their own

case). `attorney` additionally requires a shared invite code (constant-time compared), is separately rate limited, can be disabled entirely by configuration, and the application **refuses to boot in production** if the code is still the default. Submitting the public form still requires no account.

- Uploads are hardened as described in Section 6.1.

- The application refuses to start in a production environment with an insecure signing key, a default seed password, or debug mode enabled.

9. Observability and operations

- **Structured JSON logs** with timestamp, level, logger name, message, request

id, and per-event fields.

- **Access-log middleware** recording method, path, status, and duration for every request.
- **Request id** generated or propagated via `X-Request-ID`, present in every log line and every response and error body.
- **Probes:** liveness and readiness endpoints (readiness checks the database) for load balancers and orchestrators; the container image also defines a health check.
- **Security headers** on every response: content-type options, frame options, referrer policy, cross-origin-opener policy, permissions policy.

10. Design decisions and trade-offs

Each decision below lists the alternatives considered, the reasoning, the cost we accept, and the signal that should make us revisit it.

10.1 Next.js as a backend-for-frontend

Alternatives: browser calls the API directly with the token in local storage (readable by any cross-site-scripting payload); browser calls the API directly with a cookie set by the API (cross-origin cookie, needs relaxed same-site and strict cross-origin sharing, larger request-forgery surface); make Next.js the whole backend (discards the FastAPI service and its clean test story).

Decision and reasoning: the browser only ever talks to its own origin; route handlers attach the token server-side. The token is unreachable from JavaScript, cross-origin sharing is off the hot path, and the API can be private.

Cost: one extra network hop and a little proxy code per browser-facing endpoint; two deployables to run and version.

Revisit if: the proxy layer starts accumulating real logic, or a gateway platform takes over token custody.

10.2 A fully asynchronous FastAPI stack

Alternatives: Django (batteries included but synchronous by default and heavier than the domain needs); Flask (minimal, but re-assembling validation, serialization, and async by hand).

Decision and reasoning: the two slowest operations are uploads and database round-trips; under load a synchronous stack starves worker threads on I/O. Async-native FastAPI keeps a slow call to a parked coroutine, and gives declarative validation and API docs.

Cost: async SQLAlchemy is less mature and its lazy-loading rules are stricter (handled with explicit eager loading and flushes); "async all the way" is a discipline that a single stray synchronous database call would break.

Revisit if: we need Django's admin for operations, or the team is materially more productive in synchronous code than the load justifies.

10.3 PostgreSQL in production, SQLite in development and tests

Decision and reasoning: one connection string, two dialects. A contributor can clone and run with nothing installed, and the full test suite runs in seconds with no database container. The ORM and queries stay within the common subset; migrations are authored against Postgres.

Cost: a real dev/prod parity gap - SQLite has coarser datetime precision, looser constraint enforcement, and different locking. This has already required one workaround (setting event timestamps in code for ordering). CI therefore **also** runs the migrations against a real Postgres service.

Revisit if: the parity gap causes a second production surprise - at that point a containerized Postgres in every environment is worth the CI minutes.

10.4 The public form requires no account

Decision and reasoning: the intake form is the point of the product, so it has the lowest possible friction - no login, only rate limiting and upload hardening. Accounts are optional and additive.

Cost: an unauthenticated endpoint that writes to the database and stores files is an abuse target. In place: per-IP rate limiting, content-type allow-list, magic-byte check, bounded read with a size cap, filename sanitization. Not built: CAPTCHA, spam heuristics, virus scanning.

Revisit if: abuse appears in the logs - a CAPTCHA is the first lever.

10.5 Applicant-to-lead ownership by email, not a foreign key

Alternatives: require an account before submitting (contradicts the requirement); email a one-time claim link on submission (strongest integrity, but adds a token table, expiry handling, and puts email deliverability on the critical path).

Decision and reasoning: at submission time there is no user row to reference, and a prospect may register only later. Matching `lead.email` to the signed-in user's email at read time costs nothing at submission and works retroactively.

Cost: the model is only sound once registration verifies email ownership - which is a required production hardening, not a finished property today. There is also no delete-cascade relationship between a user and their leads; it is a runtime join.

Revisit before production: add email verification on registration - it is the precondition that makes this safe. A claim-token flow is the stronger option if ownership needs to be explicit in the data.

10.6 Lead status as an explicit, data-driven state machine

Decision and reasoning: rather than a boolean or an ad-hoc string, status is an enum plus a single transition table, and one service function is the only writer. It validates every move, fails illegal ones loudly and consistently, and writes the audit event centrally.

Cost: more ceremony than a boolean for today's two states - a deliberate bet that lead lifecycles always grow.

Revisit if: transitions ever need conditions richer than "the from-state is allowed" (a per-transition role, required fields) - then a small rule object per transition beats the table.

10.7 REACHED_OUT is reversible

Alternatives: keep it terminal and fix mistakes with a manual database edit (a "pure" model with an unusable operations story); a time-boxed undo (adds a clock to the domain and a "why is the button gone now" question).

Decision and reasoning: both states are non-destructive, and the audit trail already records who and when for every move - so a strict terminal rule bought integrity we were already getting from the event log, at the cost of a real operational hazard. Reopening clears the `reached_out_*` cache and appends a distinct event; nothing is rewritten.

Cost: "reached out" is no longer a monotonic, latching fact. A report that counts reached-out leads is a point-in-time snapshot; "was this lead *ever* contacted" becomes a query over the event log, not a column check.

Revisit if: compliance requires contact to be a latching fact - then reopening becomes its own state rather than a return to `PENDING`.

10.8 Append-only audit events in the same transaction

Decision and reasoning: "who marked this, and when?" and "what has happened to this lead?" are questions an internal tool always gets asked, and an `updated_at` column cannot answer them. Every creation and transition writes an immutable event in the same transaction as the change, so the two cannot disagree. This also powers the per-lead timeline and the cross-lead activity feed with no extra bookkeeping.

Cost: two writes per transition (negligible here) and an unbounded table (fine for years at this volume; archival is a future concern).

10.9 One error envelope with typed domain exceptions

Decision and reasoning: a frontend that has to special-case the framework's validation shape, our own 4xx shapes, and unhandled 500s grows bugs at every boundary. Instead, every non-2xx response has one shape; services raise typed, HTTP-agnostic exceptions that a single set of handlers maps to a status and the envelope.

Cost: a small taxonomy to maintain, and the discipline to raise the typed error rather than a bare framework exception.

10.10 Token in an httpOnly cookie, no refresh token

Decision and reasoning: for a small trusted user base, a single stateless access token in an `httpOnly` cookie needs nothing stored server-side and is safe from cross-site-scripting exfiltration.

Cost: no revocation - a stolen token is valid until it expires, and "log out everywhere" is not possible without a denylist. Disabling a user is enforced on every request via the active-user check, which is at database latency rather than instant. The long token lifetime used for demo convenience is the wrong default for production.

Revisit before production: a short access-token lifetime with refresh-token rotation and silent renewal; a denylist keyed by token id if immediate revocation is required.

10.11 Attorney self-registration behind an invite code

Decision and reasoning: attorneys see all lead information, so their registration path needs a gate, while applicant registration must stay open. The same endpoint serves both, branching on role. The attorney branch requires a shared, constant-time-compared code, is separately rate limited, has a kill switch, and blocks production boot on the default value.

Cost: a shared secret is a speed bump, not real access control - anyone with the code becomes an attorney.

Revisit before production: admin-issued single-use expiring invitations, or SSO against the firm's directory. The role column and the attorney dependency are already the enforcement points.

10.12 Email as a background task behind a provider interface

Decision and reasoning: email must never slow or fail a submission, and development and CI must run offline. A background task dispatched after the response, plus a provider interface with a console default, achieves both. Swapping to a real provider is one class.

Cost: the background task is in-process and not durable - an email is lost if the worker dies between sending the response and sending the mail. Acceptable at low volume and it fails safe (the lead is still recorded), but it is the first thing to outgrow.

Revisit if: volume matters or delivery must be guaranteed - move dispatch to a durable queue with retries and a dead-letter queue; the provider interface does not change.

10.13 External dependencies behind interfaces

Decision and reasoning: storage and rate limiting each sit behind an interface with the simple implementation as the default. The scale-up path is "add one class, change one selector" with no call-site churn.

Cost: the in-process rate limiter is per-instance - running several replicas multiplies the effective limit and it resets on deploy. It is correct for a single instance; a shared-state implementation is the documented fix and is required before running more than one instance. The fixed-window algorithm also allows a brief burst at a window boundary, which is acceptable for abuse-prevention.

Revisit when: we run more than one API instance.

10.14 Frontend: URL-as-state, Server Components, no data-fetching library

Decision and reasoning: list views are Server Components with filter, search, and pagination in the URL; there is no client cache to reconcile, views are shareable and back-button-correct, and less JavaScript ships. Because the fetch is server-side, the token stays server-side.

Cost: every filter change is a server round-trip (imperceptible at this volume, slow for very large lists), there are no live updates (two attorneys see each other's changes on their next navigation), and hand-rolled CSS is more upfront work than a utility framework.

Revisit if: the list grows past a few thousand rows (server-side keyset pagination) or real-time collaboration matters (server-sent events on the activity feed).

10.15 Confirmation dialogs rather than an undo/toast pattern

Decision and reasoning: marking a lead reached out, and reopening one, are consequential in a multi-user tool. A toast-with-undo needs a grace period during which the change is ambiguously pending - a problem when a second attorney may already be viewing that lead. An explicit confirm is unambiguous, and because the action is genuinely reversible in the domain, a misclick that gets past the dialog is still recoverable, with an audit entry that a toast-undo would not leave.

Cost: a second click on every transition - fine for low-frequency, high-stakes actions; friction on a bulk workflow.

Revisit if: attorneys need to action many leads at once - then batch select with a single confirmation.

11. Future work and production hardening

Area	Current state	Next step
Registration	No email verification	Verification mail plus a verified flag; this is the precondition for the email-based ownership model.
Sessions	Single long-lived token, no revocation	Short access-token lifetime, refresh rotation, optional token-id denylist.
Attorney access	Shared invite code	Admin-issued single-use invitations, or SSO.
Email delivery	In-process background task	Durable queue with retries and a dead-letter queue.
Uploads	Type, content, and size checks	Add virus scanning in the upload path.
Rate limiting	In-process, per-instance	Shared-state implementation before running more than one instance.
Lead search	Substring match on name and email	Postgres full-text search or a search index.
Dashboard	Refresh on navigation	Server-sent events on the activity feed for live updates.
Secrets	Environment variables	A managed secrets store.

12. Deployment

- A single command brings up Postgres, the API, and the web app. The API image is

multi-stage and runs as a non-root user with an init process and a container health check; its entrypoint waits for the database, applies migrations, seeds the initial attorney, and serves under a production server with worker processes. The web app is built as a standalone bundle.

- Configuration follows twelve-factor conventions: every setting has a safe local

default and a documented environment variable.

- Suggested production topology: managed Postgres, the API as several stateless

replicas behind a load balancer, object storage for resumes, a shared-state rate limiter, the frontend on a platform host or the same cluster, and secrets from a managed store.

13. Testing strategy

- **Backend:** pytest with async fixtures and an in-memory database. Coverage

includes the public submission (happy path, field validation, content-type spoofing, size cap, rate limit), authentication (login, current-user, bad/expired/wrong-issuer tokens, a disabled user, rate limit), the internal list/get/search/filter/pagination/download endpoints, both directions of the state machine, the audit trail, and unit tests for the content sniffer, filename sanitizer, rate limiter, email templates, and the production boot guard. A coverage floor is enforced.

- **Gates, in order:** lint and format, static type checking, a migration

up-and-down, then the test suite.

- **Frontend:** lint, type check, and a production build in CI. Most logic lives

behind the well-tested API and the thin proxy boundary.

- **Continuous integration:** a backend job (with a real Postgres service for the migration check), a frontend job, and a container-build job.